

Chat Context Export

Platform: ChatGPT

Exported: 13/09/2026, 15:23:34

Conversation ID: 6a9ec638-ed14-83ee-8750-f4090d9cf3fc

Conversation

User

Redis database setup Redis runs locally at using database . The Redis data itself is runtime state and is not stored in GitHub. The fingerprint catalog is committed to the repository in these files: - : curated JA3, JA3S, JA4, and JA4S records - : public and legacy JA3 records No manual database import is required. When the C++ engine or Python application starts, it connects to Redis and automatically seeds the committed records. A fresh clone therefore gets the same catalog after Redis is installed and running. Verify that Redis is available with: Expected output: If Redis is not running, start it with: give me the commands for mac

Code Block 1 (text)

```
bash
sudo apt update
sudo apt install -y build-essential cmake libpcap-dev libssl-dev redis-server python3-venv
sudo systemctl enable --now redis-server
```

Code Block 2 (text)

```
bash
redis-cli ping
```

Code Block 3 (text)

```
text
PONG
```

Code Block 4 (text)

```
bash
sudo systemctl start redis-server
```

Assistant

On macOS, you don't use or . Since you're on Apple Silicon, the easiest setup is through Homebrew. 1. Install the dependencies For the C++ compiler/build tools, install Apple's Command Line Tools: If they're already installed, macOS will tell you. 2. Start Redis Instead of , Homebrew uses : Check that Redis is

running: Expected: 3. Python virtual environment macOS usually already has Python 3, but I'd recommend Homebrew Python: Then: 4. Redis database setup You don't need to manually import anything, exactly as in your Linux instructions. Your application should seed: into: You can verify the Redis server: And check the database: Then: Exit with: 5. Useful macOS equivalent commands LinuxmacOS Complete copy-paste setup One important difference for your TLS fingerprinting C++ project: on macOS, and OpenSSL installed by Homebrew may not be found automatically by CMake. If your build later gives errors like or , you'll likely need to point CMake at Homebrew's prefixes.

Code Block 1 (text)

```
brew update  
brew install cmake libpcap openssl redis
```

Code Block 2 (text)

```
xcode-select --install
```

Code Block 3 (text)

```
brew services start redis
```

Code Block 4 (text)

```
redis-cli ping
```

Code Block 5 (text)

```
PONG
```

Code Block 6 (python)

```
brew install python
```

Code Block 7 (python)

```
python3 -m venv .venv  
source .venv/bin/activate
```

Code Block 8 (text)

```
code/db/seed_fingerprints.json  
code/python/fingerprints.json
```

Code Block 9 (text)

```
127.0.0.1:6379  
database 0
```

Code Block 10 (text)

```
redis-cli ping
```

Code Block 11 (text)

```
redis-cli
```

Code Block 12 (text)

```
SELECT 0  
DBSIZE
```

Code Block 13 (text)

```
exit
```

Code Block 14 (text)

```
# Homebrew  
brew update  
  
# Build/runtime dependencies  
brew install cmake libpcap openssl redis python  
  
# Apple compiler/toolchain  
xcode-select --install  
  
# Start Redis  
brew services start redis  
  
# Verify Redis  
redis-cli ping  
  
# Python environment  
python3 -m venv .venv  
source .venv/bin/activate
```